2026년『제2회 내 칩(My Chip) 경진대회』심사 자료

■ 경진대회 신청번호 :

■ 참여 팀명 : 플러팅

■ 참여 학생(팀원) 정보

성명	노지우	최성우	전병찬	백민재
학교	광운대학교	광운대학교	광운대학교	광운대학교
학과(학년) *학부/석사/박사 표기	컴퓨터정보공학부, 학부 4학년	컴퓨터공학과, 석사 1학년	컴퓨터정보공학부, 학부 4학년	컴퓨터정보공학부, 학부 4학년
연락처(Mobile)	010-8541-3379	010-8835-8911	010-6358-2947	010-4299-1679
E-mail	avo0317@naver.com	ukleon123@gmail.com	cchhann01@gmail.com	alstjwo98@naver.com
『내 칩 제작서비스』 참여 차수 및 신청번호	N20260602343	N20260602343	N20260602343	N20260602343
『내 칩 제작서비스』 참여 당시 학과(학년)	컴퓨터정보공학부, 학부 4학년	컴퓨터정보공학부, 학부 4학년	컴퓨터정보공학부, 학부 4학년	컴퓨터정보공학부, 학부 4학년

■ 지도교수 정보

성 명	이성원	학교	광운대학교
		학과	컴퓨터정보공학부
연락처	010-9472-2751	E-mail	swlee.kw@gmail.com

■ 설계명: An Area-Efficient FP8 Mixed-Precision Neural Node for CNN Inference

■ 요약문

본 연구에서는 CNN 추론을 임베디드 시스템에서 수행하기 위해, FP8 혼합 정밀도로 양자화된 파라미터를 직접 처리하는 하드웨어 뉴런을 설계하고 MPW 0.5um 공정으로 제작하고 검증하였다. 제안 설계는 E4M3와 E5M2 두가지 FP8 포맷을 동시에 입력으로 받아 내부 공통 포맷인 E5M3로 변환하여 하나의 데이터패스로 처리한다. 곱셈 결과는 가수를 확장한 E5M7포맷으로 누산하여 오차의 누적을 줄이고, 출력 단에서는 다시 FP8포맷으로 변환한다. 3x3 컨볼루션 커널에 해당하는 9개 원소의 내적은 하나의 누산기로 9개 사이클에 걸쳐 순차 처리하여 면적을 줄였다.

RTL설계부터 논리합성, P&R(배치&배선), STA(static timing analysis), GDSII 생성까지 전 과정을 수행하였고, 최종 회로는 1,313개의 표준 셀로 구현되었다. 제작한 칩은 functional/yield(production) test를 위해 183,877개 테스트 벡터를 제작하였고, 검증 보드에서 테스트벡터를 칩으로 입력하였다. 그 결과 모든 출력값이 기대값(RTL 출력값)과 사이클 단위로 일치하였으며, 설계 목표 주파수 20Mhz 대비하여 최대 동작 주파수는 42MHz로 확인되었다. 토글 커버리지(칩 내부 TR/net를 얼마나 빠짐없이 토글하였는가)는 97.8%를 달성하였다.

■ 회로 설계의 목적 및 필요성

CNN 은 영상 인식 분야에서 높은 정확도를 보이지만, 수백만 개의 파라미터와 대량의 곱셈·누산 연산을 요구한다. 메모리 용량, 전력, 칩 면적이 제한된 임베디드 시스템에서 32 비트 부동소수점 파라미터를 그대로 저장하고 연산하기는 어렵기 때문에, 파라미터를 낮은 비트 폭으로 표현하는 양자화가 필수적이다.

8 비트 정수(INT8) 양자화는 하드웨어가 단순하지만 표현 범위가 좁아[3], 레이어마다 분포가 다른 가중치와 활성화 값을 함께 표현하기 어렵다. 반면 8 비트 부동소수점(FP8)은 지수를 통해 넓은 동적 범위를 확보할 수 있으며, 정밀도가 높은 E4M3 와 지수 비트가 많아 범위가 넓은 E5M2 로 나뉜다[1]. 따라서 데이터의 분포에 따라 두 포맷을 골라 쓰는 혼합 정밀도[2] 양자화는 임베디드 환경에서 정확도와 효율을 함께 얻을 수 있는 방법이다.

그러나 양자화 기법은 이를 그대로 실행할 수 있는 하드웨어가 있어야 효과를 얻을 수 있다. 임베디드용 범용 프로세서는 대부분 FP8 연산을 지원하지 않아 소프트웨어 변환이 필요하고, 포맷별 연산기를 따로 두면 면적이 늘어난다. 따라서 양자화된 파라미터를 직접 받아 가중합, 활성화, 재양자화까지 수행하는 작은 면적의 하드웨어 뉴런이 필요하다.

이러한 필요성을 바탕으로, 본 연구에서는 E4M3 와 E5M2 로 양자화된 파라미터를 하나의 데이터패스로 처리하는 혼합 정밀도 하드웨어 뉴런을 구현하고, 이를 실제 칩으로 구현하여 검증까지 진행한다.

본 하드웨어 뉴런은 여러 개를 배열하여 임베디드용 CNN 가속기의 처리 요소(PE)로 확장할 수 있다.

■ 설계 결정사항

본 절에서는 설계 과정에서 내린 주요 결정 사항을 설명한다.

1. Floating point 타겟 포맷

본 설계가 지원하는 부동소수점 형식은 E4M3와 E5M2이다.

NVIDIA에서 낸 논문 “FP8 FORMATS FOR DEEP LEARNING”에서 정한 FP8의 포맷을 따르되, 특수값의 처리는 면적 감소를 위해 일부 변형하였다.

본 설계에서 사용한 FP8의 binary format은 표 1에 명시하였다.

	E4M3	E5M2	E5M3
Exponent bias	7	15	15
Infinities	S.1111.110	S.11110.11	S.11111.000
NaN	S.1111.111	S.11111.01	S.11111.001
Zeros	S.0000.000	S.00000.00	S.00000.000
Max normal	S.1111.110 $= 1.75 \times 2^{8} = 448$	S.11110.11 $= 1.75 \times 2^{15} = 57{,}344$	S.11110.111 $= 1.875 \times 2^{15} = 61{,}440$
Min normal	S.0001.000	S.00001.00	S.00001.000

표 1. 본 설계에서 사용한 FP8의 binary format

2. 내부 연산 포맷: E5M3 / E5M7

2.1 곱셈 포맷: E5M3

본 설계에서는 두가지 포맷 E5M2와 E4M3를 각각 동시에 받아 처리하는 것을 목표로 한다.

해당 기능을 구현하기 위해, E5M2와 E4M3 두가지 포맷을 받아 내부적으로 E5M3 포맷으로 확장하여 곱셈을 수행한다. 그림 1은 이 변환 과정을 나타낸다. E5M3의 binary format은 표 1에 명시하였다.

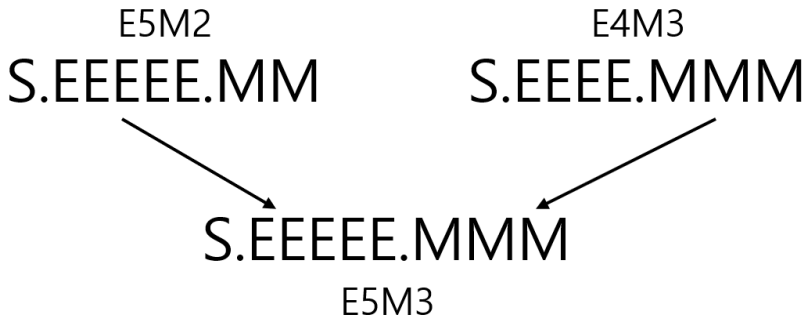


그림 1. E5M2와 E4M3의

혼합 정밀도 곱셈을 위한 E5M3 구조

2.2 누산 포맷: E5M7

ACC(Accumulator)를 통해 연속적인 누산을 진행할 때에는 오차의 누적을 방지하지 위해 mantissa를 확장하여 E5M7 포맷을 사용하였다. 이는 면적-정확도 trade off에서 허용 가능한 오차 범위 내 가장 작은 크기의 포맷을 선택한 것이다.

3. 데이터 포맷

MyChip 으로 제작할 수 있는 칩의 핀 개수는 28 개이다. 하지만 병렬 데이터 전송을 위해서는 weight 와 input 을 bit 단위로 동시에 입력되어야 하는데, 3x3 커널의 경우 weight 9 개 x 8bit = 72 이다. Input 도 동일하게 72 개의 핀이 필요하다. 따라서 핀 수의 제약을 충족하기 위해, 병렬 데이터 전송 방식이 아닌 serial 전송으로 변경하였다. Serial 로 전송을 할 경우 병목이 발생하지만, 비교적 고속으로 데이터를 전송하여 해당 병목을 완화하고자 하였다. Serial 통신 종류는 추후 설계를 원래 목적과 맞게 병렬 전송으로 변경하여 제작할 수 있도록 SPI 를 선택하였다.

■ 회로 설계 내용

본 절에서는 회로의 control signal 구성, 모듈 설계, 데이터 플로우와 같은 architecture를 설명한다. MyChip 공정은 칩 면적과 핀 수에 제약이 있어, 일부 성능과 기능을 축소·변경하여 면적을 줄였다. 이러한 설계상 절충 사항은 다음과 같으며, 각 항목은 이어지는 모듈 설명에서 자세히 다룬다.

기능-면적 trade off로 인한 설계 제한

1. FPU와 adder 개수 제한
2. Serial 통신(SPI) 사용
3. Register file구현 시 latch 사용
4. 특수 값 처리 단순화

전체적인 data flow는 그림 2와 같이 구성 되어있다.
그림 2의 하단에 각 모듈의 세부 명칭을 명시해두었다.

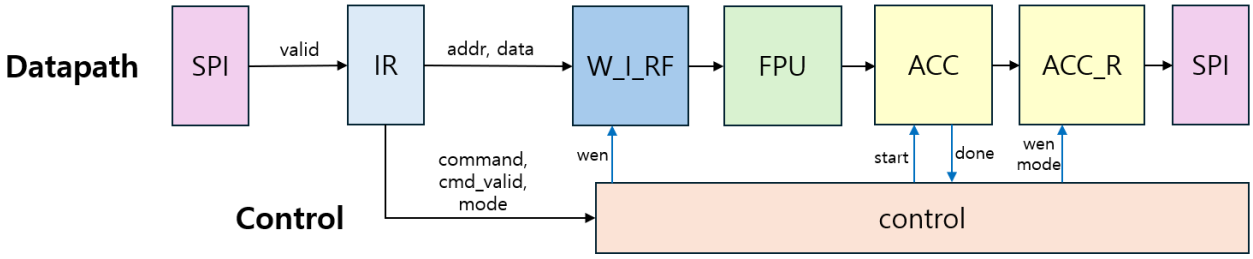


그림 2. Data flow diagram

SPI: Serial Peripheral Interface
IR: Instruction Register
FPU: Floating Point Unit
ACC: ACCumulator
ACC_R: ACC Register

그림 2의 각각의 블록은 개별 Verilog 모듈로 구성하였다. 지금부터 각 블록의 기능과 역할에 대해 서술한다.

1. SPI

기본적으로 CNN 연산 가속을 위해서는 데이터를 병렬로 받고, 병렬로 연산하여 병렬로 출력해야한다.

하지만 28핀이라는 한계로 인해 데이터를 병렬로 받을 수 없었고, 따라서 SPI 통신을 사용하여 데이터를 주고받도록 설계하였다.

SPI 통신을 구현할 때, CDC(Clock Domain Crossing)문제가 발생할 수 있다. CDC란 서로 다른 클럭으로 동작하는 회로 사이에서 신호가 오가는 것을 말한다.

본 설계에서는 SPI 마스터가 만든 SCLK와, 그에 맞춰 바뀌는 MOSI, SSn 신호가 칩 내부 system clock과 관계없이(비동기로) 들어온다. 플립플롭은 클럭 에지 전후의 setup/hold time 동안 입력이 변하지 않아야 값을 올바르게 저장한다.

그런데 비동기 신호는 언제 바뀔지 알 수 없어서 이 조건을 지키지 못하는 순간이 생길 수 있다. 그러면 플립플롭 출력이 0도 1도 아닌 중간 전압에 머무르는 준안정 상태(metastability)에 빠질 수 있다. 이 값을 뒤쪽 로직들이 서로 다르게 해석하면 데이터 비트가 빠지거나 중복되는 오동작이 생긴다.

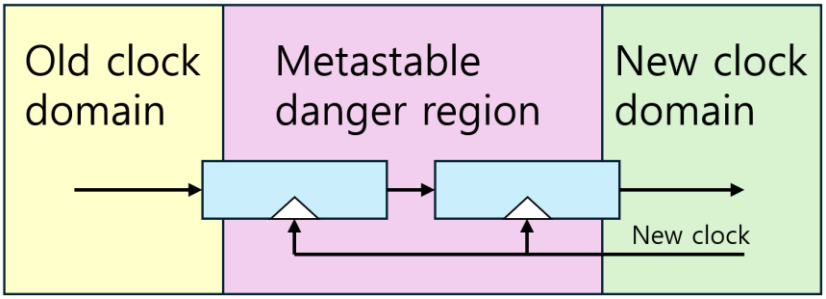


그림 3. CDC 문제의 해결책인 Two flip-flop

해당 문제가 일어나는 것을 방지하기 위해, SPI clock을 system clock이 샘플링해서 사용하는 방식으로 구현하였고, 그림 3에서 해당 방법을 나타내고 있다.

이 방식에서는 SCLK를 클럭으로 쓰지 않고, system clock으로 샘플링하는 일반 데이터 신호로 다룬다. 비동기 입력은 system clock으로 동작하는 플립플롭을 두 단 연속으로 거친다. 첫 단이 준안정 상태에 빠지더라도 다음 클럭 에지까지 한 주기 동안 안정된 값으로 수렴할 시간이 생기므로, 뒤쪽 로직으로 준안정 상태가 전파될 확률이 크게 줄어든다. 동기화된 이후에는 칩 내부의 모든 레지스터가 system clock 하나로만 동작하는 단일 클럭 도메인이 된다.

IR(Instruction Register)

IR 모듈에서는 E5M2와 E4M3를 E5M3로 확장하고, SPI 모듈에서 받아온 명령어를 각 필드별로 디코딩한다.

SPI 로 받아오는 데이터의 각 필드는 그림 4와 같이 구성 되어있다.

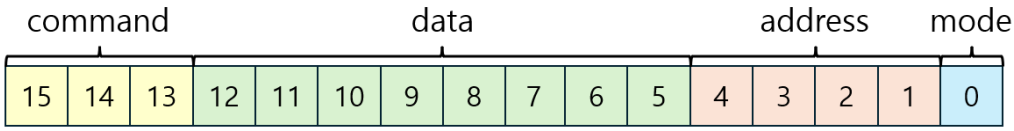


그림 4. SPI 로 받아오는 data field

각 필드가 나타내는 값은 다음과 같다.

command: load weight, load input, compute, acc, read result
data: E5M2/E4M3 data
address: 0~8 register file number
mode: E5M2/E4M3 mode selection

E5M2를 E5M3로 확장할 때는 mantissa의 LSB에 0을 붙여서 확장하고, E4M3를 E5M3로 확장할 때는 bias 보정을 통해 지수를 맞추어 준다.

2. W_I_RF(Weight, Input Register File)

W_I_RF에서는 각각 weight와 input을 저장한다.

Command가 load weight일 경우, weight RF를 선택하고, 해당 RF안에서 command와 함께 전송된 address에 맞는 register에 weight를 저장한다.

W_I_RF를 구현할 때, 처음에는 DFF를 사용하였으나 칩의 면적 절감을 위해 latch를 사용하는 것으로 변경하였다. DFF에서 latch로 변경하여 얻은 면적 절감 효과는 그림 5에서 나타내었다.

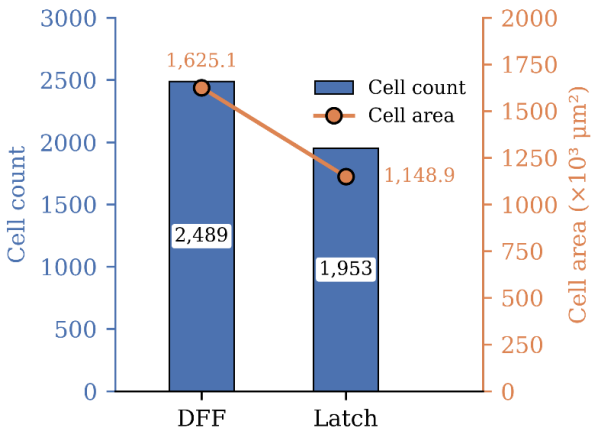


그림 5. DFF 또는 latch를 사용한 RF의 면적 비교

3. FPU(Floating Point Unit)

FPU는 W_I_RF에 저장된 weight와 input을 한 쌍씩 받아 부동소수점 곱셈을 수행하는 모듈이다. IR에서 E4M3와 E5M2 입력을 모두 E5M3로 확장하여 저장하므로, FPU는 mode와 관계없이 하나의 E5M3 × E5M3 곱셈기로 E4M3, E5M2 및 두 포맷이 섞인 혼합 정밀도 곱셈을 모두 처리한다. 곱셈 결과 역시 E5M3(9bit) 포맷으로 출력한다.

FPU는 클럭과 start 신호 없이 순수 조합회로로 구현하였다. 따라서 입력이 바뀌면 같은 사이클 안에 결과가 확정되며, 별도의 제어 신호나 파이프라인 레지스터가 필요 없다. FPU의 내부 구조는 그림 6과 같다.

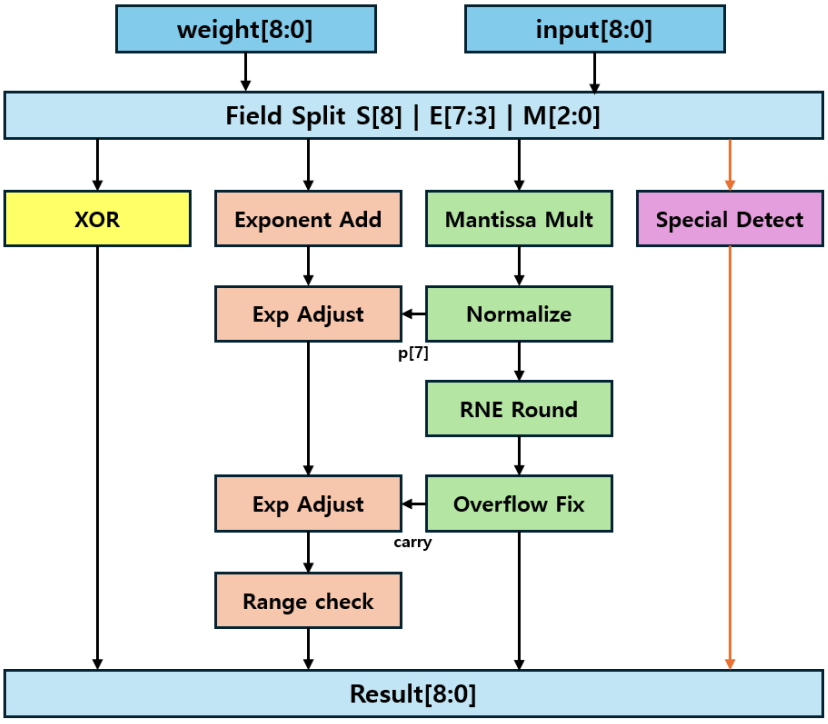


그림 6. FPU 내부 블록도

FPU 개수 축소

3×3 커널의 9개 weight-input 쌍을 원소마다 별도의 FPU로 처리하면 9개의 FPU가 필요하다. 그러나 칩의 사이즈 제한으로 인해 FPU를 3개로 축소하고, ACC 또한 1개로 축소하여 순차적인 누산이 수행되도록 설계하였다.

그림 8과 같이 9개의 원소를 {0,1,2}, {3,4,5}, {6,7,8}의 세 그룹으로 나누어 FPU0, FPU1, FPU2가 각각 한 그룹을 담당한다. 각 FPU 앞에는 그룹 내 원소 하나를 고르는 3:1 입력 MUX를, FPU 출력에는 세 FPU 중 하나의 결과를 고르는 3:1 출력 MUX를 두었다. 두 MUX의 선택 신호는 ACC의 누산 카운터(0~8)를 인덱스로 하는 고정 스케줄 테이블에서 생성되며, 매 사이클 원소 0부터 8까지 고정된 순서로 곱셈 결과 하나가 ACC에 전달된다.

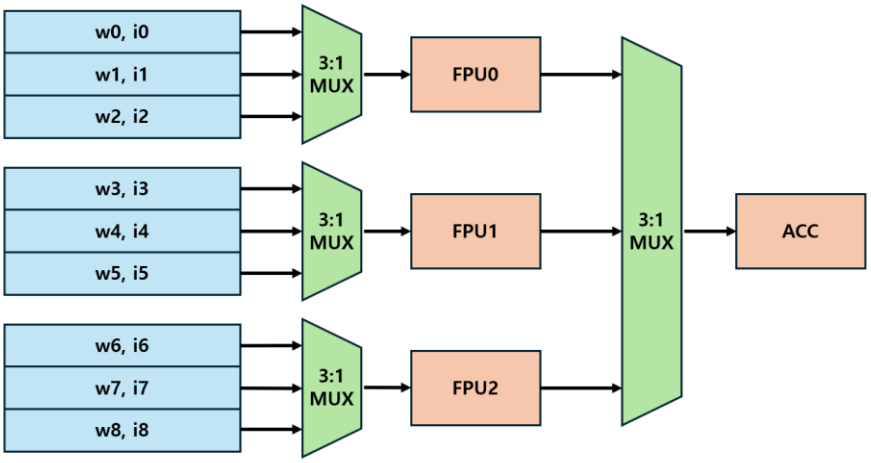


그림 8. FPU 3개를 사용한 곱셈 구조

4. ACC(Accumulator)

ACC는 FPU에서 순차적으로 전달되는 9개의 곱셈 결과를 누산하여 3×3 합성곱 한 번의 결과를 만드는 모듈이다. FPU 개수 축소에서 설명한 바와 같이 adder를 1개만 두고, 매 사이클 곱셈 결과를 하나씩 9사이클에 걸쳐 순차적으로 누산한다.

FPU의 출력은 E5M3이지만, ACC 내부에서는 설계 결정사항에서 정한 E5M7 포맷으로 누산한다. 두 포맷은 지수 폭(5bit)과 bias(15)가 같으므로, 가수 하위에 4bit의 0을 붙이는 것만으로 별도의 연산 없이 E5M3 값을 E5M7로 손실 없이 확장할 수 있다.

5. ACC_R(ACC Register)

ACC_R는 E5M7(13bit) 누산 결과를 사용자가 선택한 출력 포맷인 E4M3 또는 E5M2(8bit)로 변환하고, ReLU를 적용한 뒤 레지스터에 저장하는 모듈이다. 출력 포맷은 read result 명령과 함께 전송된 mode 비트로 결정되며, 저장된 8bit 결과는 SPI에서 상위 8bit를 0으로 채운 16bit 데이터로 마스터에게 전송된다.

ReLU 및 출력 레지스터

CNN에서 합성곱 뒤에 이어지는 ReLU 활성화 함수를 ACC_R에 통합하였다. 누산 결과의 부호 비트가 1(음수)이고 NaN이 아니면 포맷 변환 결과와 관계없이 0을 저장한다.

출력 레지스터는 CONTROL의 write enable(wen) 신호가 들어온 사이클에만 변환 결과를 저장하고, 그 외의 사이클에는 0으로 초기화된다. CONTROL은 read result 명령을 받으면 wen을 1사이클 동안 올리고 바로 다음 사이클에 SPI의 tx_load를 올리도록 순서를 맞추어, 결과가 레지스터에 유효한 사이클에 SPI 송신 버퍼로 옮겨지도록 하였다.

■ 측정 방법, 결과 및 고찰

본 절에서는 제작된 칩의 검증 방법, 테스트 벡터의 구성과 생성 과정, 측정 환경 및 결과를 설명한다.

칩의 검증은 다음 3개 부분으로 진행하였다.

테스트 벡터 생성: 각 functional test, yield test를 위한 테스트 벡터와 기대값을 제작한다.

검증 보드 제작: 테스트벡터를 DUT(MyChip)로 넣을 수 있는 플랫폼을 제작한다.

Functional / Yield(production) test: RTL과 칩의 동작이 일치하는지 확인하는 functional test와 칩의 모든 부분(TR)이 정상적으로 동작하는지 확인하는 yield(production) test를 진행한다.

이때 functional/yield test는 하나의 테스트 벡터로 구성하여 검증했다.

1. 테스트 벡터 생성

테스트벡터 1개는 칩의 system clock 1 사이클에 해당하며, 칩의 입력 4비트로 구성된다.

각 동작을 나타내는 SPI 명령 1개는 16bit로 이루어져 있으며, 이전 회로 설계 내용 그림 4에 나타낸 바 있다.

SPI의 SSn을 low로 내린 뒤, 16bit 명령어 1개를 전송하고, SSn을 다시 high로 올리는 시퀀스를 '프레임'이라고 한다. 하나의 프레임 사이클을 정하기 위한 구성요소는 K와 G가 있다.

K는 SCLK의 반주기의 system clock 개수이고, G는 SSn high 구간의 system clock 개수이다.

하나의 프레임은 2K + (16 x 2K) + G 개의 사이클로 구성된다. 검증을 진행할 때는 K=4, G=8로 세팅하여 진행하였다. 이때 K=4로 지정한 이유는 보충자료 **(a) 테스트 벡터 K=4로 지정한 이유**에서 설명한다. 테스트 벡터는 Python으로 작성한 벡터 생성기로 만들었다. 생성기는 SPI 프레임, MAC 연산, 리셋을 함수 단위로 정의하고 이를 조합하여 케이스별 시나리오를 구성한다.

기대값은 벡터 생성기가 직접 계산하지 않고, post-layout 넷리스트를 시뮬레이션하여 얻었다. 테스트벡터를 입력으로, RTL과 GLS 출력이 모든 사이클에서 완전히 일치하는 것을 확인하였다.

전체 벡터는 검증 목적별로 17개의 케이스로 나누어 구성하였으며, 총 183,877개의 벡터를 제작하였다. 각 케이스의 세부 검증 내용은 보충자료 **(b) 테스트 벡터 케이스별 세부 검증 내용**에 첨부한다.

테스트 벡터는 functional과 yield(production) test를 모두 진행하기 위한 것으로, 각 17개의 케이스로 function을 검증하고, 칩 내부의 net를 최대한 빠짐없이 1번 이상 스위칭시켜 최대 coverage를 달성하도록 제작하여 yield(production) test를 진행하였다.

coverage는 테스트 벡터가 칩 내부의 트랜지스터를 얼마나 빠짐없이 스위칭시키는지를 나타내는 지표이다. 하지만 게이트 수준 넷리스트에서는 트랜지스터의 상태를 직접 관찰할 수 없으므로, 셀 사이를 연결하는 net의 토글 여부를 기준으로 측정하였다.

본 칩은 정적 CMOS 표준 셀로 구성되어 있으며, 각 트랜지스터의 게이트는 셀 입력 핀, 즉

넷리스트의 net에 연결된다. 따라서 어떤 net이 0→1과 1→0으로 천이하면, 그 net을 게이트 입력으로 받는 모든 NMOS와 PMOS가 각각 on과 off 상태를 한 번 이상 거치게 된다. 또한 셀 출력 net이 양방향으로 천이하였다는 것은 해당 셀의 풀업(PMOS) 경로와 풀다운(NMOS) 경로가 모두 켜진 적이 있음을 의미한다. 이러한 이유로 net의 양방향 토글 여부를 트랜지스터 스위칭 여부의 대리 지표로 사용하였다.

본 테스트 벡터는 토글 coverage 97.8%를 달성하였으며, 이는 테스트 벡터 실행 중 모든 net의 VCD를 기록하여 토글 여부를 확인한 것이다. 그 결과 전체 1,383개 net 중 1,352개가 0 → 1, 1 → 0 양방향으로 모두 천이하였다.

토글되지 않은 31개의 net은 대부분 설계상 고정된 값(VDD, VSS, ReLU이후 부호 등)이었고, 실제로 토글되지 않은 net은 17개에 불과한 것을 확인하였다.

2. 검증 보드 제작

검증을 위한 보드는 그림 10과 같은 구조로 제작하였다.

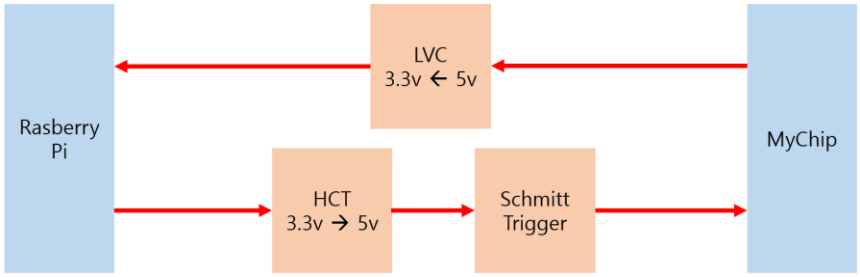


그림 10. 검증 보드 구조

측정 장비로는 Raspberry Pi Pico 2 W를 사용하였다. 칩은 5V로 동작하고 Pico는 3.3V로 동작하므로, 입출력 경로에 버스 트랜시버를 두었으며, 깨끗한 입력을 주기 위해 슈미트 트리거를 추가하였다.

실제 제작한 보드 모습은 그림 11과 같다. 칩과 TTL의 배치는 그림 10과 유사하다.

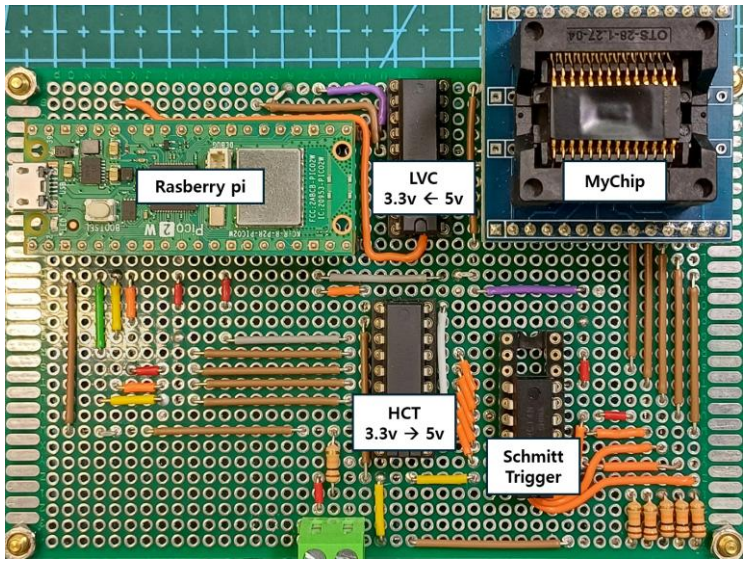


그림 11. 검증 보드

3. Functional / Yield(production) test

제작한 183,877개의 테스트벡터는 검증보드를 사용하여 MyChip으로 입력하였으며, 모든 칩(5개)이 모든 벡터에서 기대값을 만족하는 것을 확인하였다.

최대 system clock은 42MHz까지 동작하는 것을 확인하였으며, 더 높은 주파수에서는 일부 테스트벡터에서 기대값과 다른 값들이 관찰되었다.

이는 합성 타겟 주파수인 20MHz보다 약 2배 높은 주파수인데, 합성 시 타겟 주파수는 최악의 상황을 가정하였을 때 동작할 수 있는 주파수로, 실제 칩이 동작하는 대부분의 경우 타겟 주파수보다 더 높게 동작하는 것이 일반적이다.

■ 결론

본 연구에서는 CNN을 임베디드 시스템에 적용하기 위한 FP8 혼합 정밀도 양자화 기법에 적합한 하드웨어 뉴런을 설계하고, MPW 공정으로 제작하여 실측까지 검증하였다. 그 결과 서로 다른 두 FP8 포맷으로 양자화된 파라미터를 하나의 연산 구조에서 처리할 수 있었으며, 8비트 파라미터를 사용하면서도 가중합 과정의 정확도를 확보할 수 있었다. 또한 활성화와 재양자화를 뉴런 내부에서 함께 수행함으로써 출력이 별도의 후처리 없이 다음 레이어로 이어질 수 있게 되었고, 이러한 기능을 면적이 제한된 임베디드 환경에 적합한 작은 규모의 회로로 구현할 수 있었다. 아울러 설계부터 칩 제작, 실측에 이르는 전 과정을 직접 수행하여 실제 칩에서 오류 없는 동작을 확인하였으며, 이를 통해 제안한 하드웨어 뉴런의 기능적 정확성을 실리콘 수준에서 입증할 수 있었다. 본 연구의 결과는 혼합 정밀도 양자화 기법이 임베디드 하드웨어에서 실제로 구현 가능함을 보여주며, 향후 본 뉴런을 여러 개 배열하여 임베디드용 CNN 가속기의 핵심 연산 단위로 확장할 수 있을 것으로 기대된다.

■ 참고문헌

[1] P. Micikevicius et al., "FP8 formats for deep learning," arXiv preprint arXiv:2209.05433, 2022.

[2] X. Sun et al., "Hybrid 8-bit floating point (HFP8) training and inference for deep neural networks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), Vancouver, Canada, 2019, pp. 4900–4909.

[3] H. Shen, N. Mellempudi, X. He, Q. Gao, C. Wang, and M. Wang, "Efficient post-training quantization with FP8 formats," in Proc. Machine Learning and Systems (MLSys), vol. 6, 2024, pp. 483–498.

보충자료 (Supplement)

(a) 테스트 벡터 K=4로 지정한 이유

하나의 프레임은 2K + (16 x 2K) + G 개의 사이클로 구성된다. 검증을 진행할 때는 K=4, G=8로 세팅하여 진행하였다. K = 4로 정한 것은 여러 이유가 있는데, 만약 클락과 비동기로 입력이 들어오는 경우(현실에서는 대부분 비동기이다), system clock의 엣지에서 SCLK가 들어온다면 setup time과 hold time을 만족하지 못하고 metastable 상태에 빠져버린다. 이는 그림 9의 (a)가 나타낸다.

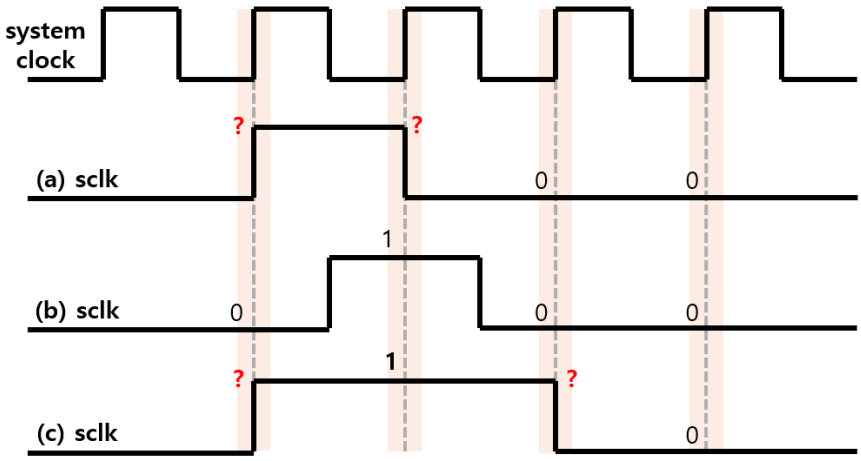


그림 9. System clock과 비동기인 SCLK

(b)에서는 운 좋게 setup/hold time에 겹치지 않게 입력이 들어와 제대로 샘플링이 되는 경우이다.

(a)와 같은 경우를 피하기 위해, 최소한 1번 제대로 샘플링이 될 수 있도록 (c) 와 같이 K=2 이상으로 설정한다.

K=4로 설정한 또 다른 이유는, MISO때문이다. SPI에서는 SCLK가 low로 토글된 이후 MISO를 새 비트로 바꾸고, master(검증 보드)에서는 SCLK가 high로 토글된 직후 MISO를 읽는다. 이때 SPI에서 SCLK는 2개의 레지스터를 거쳐 값이 전달되므로, slave(DUT)는 SCLK의 토글을 감지하는데 2사이클의 지연이 발생한다. 이는 앞부분 회로 설계 내용의 그림 3에서 설명한 바 있다.

이때 SCLK가 그림 9와 같이 비동기로, system clock의 setup/hold time에 겹쳐서 토글된다면 최악의 경우 SCLK의 토글을 감지하는데 3사이클이 걸린다. 따라서 마진을 추가하여 K(SCLK의 반주기)를 4로 지정하였다.

하나의 프레임을 구성하는 system clock 개수인 2K + (16 x 2K) + G에서 2K는 SCLK 한 사이클의 system clock 개수이고, 16bit의 명령이 SCLK 주기마다 전송되므로 16 x 2K로 설정되었다.

이때 K=4, G=8로 설정하였으므로, 하나의 프레임이 전송되는데 소요되는 system clock은 144 사이클이다. 즉 하나의 명령을 보내기 위해 144개의 벡터가 사용되는 것이다.

(b) 테스트 벡터 케이스별 세부 검증 내용

No.	케이스	벡터 수	검증 목적
0	리셋 직후 대기 상태 출력 확인	44	리셋이 끝난 뒤 명령이 없을 때 칩이 출력 핀에 의도하지 않은 값을 내보내지 않는지 확인
1	연산 전 SPI 읽기 초기값 확인	156	아직 연산하지 않은 상태에서 SPI로 읽으면 초기값 0이 나오는지 확인 (통신 경로의 기본 동작)
2	누산 없이 결과 읽기 명령 전송	2,900	누산을 하지 않은 상태에서 결과 읽기 명령이 오면 무시되는지 확인 (잘못된 명령 순서에 대한 보호)
3	전 원소 1.0 기본 MAC 연산	3,068	데이터 적재, 곱셈, 누산, 출력 변환, 결과 전송까지 전체 연산 흐름이 처음부터 끝까지 정상 동작하는지 확인
4	원소 하나씩만 값을 넣은 단독 연산 (9회)	27,516	9개 원소를 하나씩만 값을 넣어 연산해서, 각 저장 위치와 해당 곱셈 경로가 다른 원소와 섞이지 않고 독립적으로 동작하는지 확인
5	저장값 덮어쓰기 후 연산 재실행	4,284	이미 저장된 값을 새 값으로 바꿨을 때 정확히 반영되는지, 같은 연산을 다시 실행할 때 이전 누산값이 남지 않고 처음부터 다시 계산되는지 확인
6	결과 읽기 명령 연속 2회 전송	3,364	결과를 한 번 읽은 뒤 같은 명령이 다시 오면 무시되는지 확인 (결과 준비 플래그가 올바르게 지워지는지)
7	음수 포함 연산 및 ReLU 출력	9,180	음수 곱셈 결과가 올바르게 빼지는지, 양수와 음수가 정확히 상쇄되는지, 최종 합이 음수일 때 ReLU가 0을 출력하는지 확인
8	NaN·최댓값·최솟값 등 특수값 입력	12,236	NaN 입력이 결과까지 전달되는지, 표현 범위를 넘는 큰 값은 최댓값으로 고정되는지, 너무 작은 값과 지수 0 입력은 0으로 처리되는지 확인
9	반올림이 발생하는 가수 조합 연산	9,180	곱셈과 출력 변환 과정에서 잘리는 가수 비트가 규칙대로 반올림되는지, 반올림 때문에 자리가 올라가는 경우에도 지수가 올바르게 보정되는지 확인
10	크기 차이가 큰 값끼리의 누산	3,068	크기 차이가 아주 큰 두 값을 더할 때 작은 값의 자리 맞춤과 버림이 올바르게 처리되는지 확인
11	입출력 포맷(E4M3/E5M2) 조합별 연산	18,348	입력과 출력의 포맷(E4M3/E5M2) 네 가지 조합이 모두 정상 동작하는지, E5M2에만 있는 무한대 입력과 포화 처리가 맞는지 확인
12	무작위 데이터 20세트 연산	61,132	사람이 고른 값이 아닌 무작위 데이터로 연산해서, 일반적인 입력에서도 결과가 정확한지 확인
13	최고 속도 SPI 통신	1,525	SPI 신호가 칩이 허용하는 가장 빠른 속도로 들어와도 명령을 놓치지 않고 받는지 확인 (통신 타이밍 여유)

14	저속 SPI 통신	12,140	SPI 신호가 매우 느리게 들어와도 같은 결과가 나오는지 확인 (속도가 다른 외부 장치와의 호환성)
15	정의되지 않은 명령 입력	3,500	정의되지 않은 명령이나 비정상적인 신호 레벨이 들어와도 내부 상태가 바뀌지 않고 이후 연산이 정상인지 확인
16	전 원소 음수 입력 연산	12,236	9개 원소 모든 위치에서 음수 입력의 부호가 올바르게 저장·계산되는지, 음수 연산 후 양수 연산으로 돌아왔을 때 이전 부호가 남지 않는지 확인

(c) 최종 GDS 모습

